Horizon : validation corrélée de la relation de plages

Supplément algébrique et protocole expérimental

Dossier technique du dépôt ZK-agentiK

17 octobre 2026 Version expérimentale 1

Abstract

Ce supplément relie la comparaison entière d'un montant et d'un plafond aux deux engagements dérivés vérifiés par Horizon. Il donne une démonstration conditionnelle et un protocole de corrélation entre calcul JavaScript BigInt, oracle MATLAB/GNU Octave et exécution réelle de Bulletproofs en Rust. Les conditions de succès distinguent refus applicatif et rejet cryptographique. Des essais finis, même concordants, ne prouvent ni l'absence de défaut, ni la sécurité de la primitive. La source numérique est portable ; le runtime effectivement exécuté doit être lu dans chaque artefact. Aucune exécution MATLAB n'est déduite d'un résultat Octave. Le profil n'est pas post-quantique.

1 Énoncé : relation arithmétique et politique

Le profil examiné est `horizon-range-v1`, dans `labs/horizon/proof/src/lib.rs`. Ce supplément accompagne le manuscrit `docs/whitepaper/whitepaper.tex` sans le remplacer. La base de la campagne est le commit `5cf0ee2` ; chaque exécution doit identifier la révision effective de ses sources et ses éventuelles modifications locales.

Posons $U = 2^{64} - 1$ et $N = 2^{32}$. Le domaine d'entrée est $\mathcal{D} = \{0, \dots, U\}^2$. La décision numérique est

$$P(a, c) \iff 1 \le a \le N \quad \wedge \quad a \le c \quad \wedge \quad c - a < N. \tag{1}$$

Cette relation n'est pas simplement $a \le c$. Par exemple $(0, 0)$, $(N + 1, N + 1)$ et $(1, N + 1)$ satisfont la comparaison mais pas P. Le plafond c est un `u64` et peut atteindre $2^{33} - 1$; imposer $c \le 2^{32} - 1$ changerait la relation.

Frontière de politique. Une acceptation cryptographique exige aussi les signatures, la jointure du dossier, la devise EUR, l'exposant 2, les clés et rôles épinglés, les dates, le challenge indépendant et le mode permis. L'autorisation d'un effet exige encore révocation, époque, nonce et opération métier dans le contrôleur. L'oracle numérique ne reçoit aucun de ces éléments. Il ne peut donc pas prononcer une autorisation métier. Le plafond reste par dossier : deux montants de 60 sous un plafond de 100 n'établissent pas un budget cumulé de 100 ; une franchise, une conversion d'unité et une règle de paiement seraient des relations supplémentaires.

2 Du witness entier aux engagements

Soit $\mathbb{G}$ le groupe Ristretto d'ordre premier

$$q = 2^{252} + 27742317777372353535851937790883648493.$$

Les encodages canoniques et l'abstraction de groupe sont décrits dans RFC 9496 [3]. Les bases G, H sont les paramètres publics fixes de `PedersenGens::default()`, jamais choisis par l'appelant. Deux autorités construisent les engagements signés

$$A = aG + r_a H, \qquad C = cG + r_c H, \qquad r_a, r_c \in \mathbb{Z}_q.$$

Le vérifieur dérive les points, sans accepter ceux proposés séparément par le prover :

$$X = A - G, \qquad D = C - A. \tag{2}$$

Le witness de la preuve de plages agrégée est

$$x = a - 1, \quad d = c - a, \quad r_X = r_a, \quad r_D = r_c - r_a \pmod{q}.$$

Horizon utilise `RangeProof` de Bulletproofs 5.0.0 avec deux valeurs et 32 bits [2]. Ce chemin est une preuve spécialisée de plages ; il ne compile pas la politique entière en un circuit R1CS universel. Le mot « circuit » dans la campagne désigne cette relation cryptographique.

Lemme 1 (Équivalence entière)**.** *Pour $(a, c) \in \mathcal{D}$, $P(a, c)$ équivaut à l'existence d'entiers $x, d \in [0, N-1]$ tels que $a = x + 1$ et $c = x + 1 + d$.*

Proof. Si P est vraie, $x = a-1$ et $d = c-a$ satisfont les bornes. Réciproquement, $a = x+1 \in [1, N]$ et $c - a = d \in [0, N-1]$. Ces identités donnent aussi $c \leq 2N - 1$, sans addition susceptible de dépasser U. □

Lemme 2 (Complétude arithmétique)**.** *Si $P(a, c)$ est vraie, les ouvertures précédentes produisent exactement les points de (2).*

Proof. $xG+r_XH = (a-1)G+r_aH = A-G$. De même, $dG+r_DH = (c-a)G+(r_c-r_a)H = C-A$. Les deux valeurs sont dans les plages requises ; la complétude cryptographique demeure celle de la primitive. □

3 Correction conditionnelle : connaissance et liaison

Hypothèse 3 (Émission, primitive et implémentation)**.** *Les autorités ont construit ou contrôlé les ouvertures de A et C depuis des valeurs du domaine `u64`, avec les métadonnées appropriées. Le logarithme discret entre G et H n'est pas connu d'un adversaire efficace. La primitive fournit l'argument de connaissance des ouvertures dans les plages sous son modèle de sécurité. Encodages, paramètres, transcript, RNG et vérification implémentent fidèlement cette primitive.*

Le papier Bulletproofs et sa révision corrigée sont la référence, sans réduction ni audit logiciel réinventé ici [1]. Une simple existence mathématique d'ouvertures serait insuffisante : puisque H engendre $\mathbb{G}$, tout point admet une ouverture à toute valeur scalaire pour un masque approprié. Le passage à une valeur attestée exige la connaissance extractible et la liaison calculatoire.

Lemme 4 (Deux ouvertures incompatibles)**.** *Si $vG + rH = v'G + r'H$ et $v \not\equiv v' \pmod{q}$, alors*

$$\log_G H = (v - v')(r' - r)^{-1} \pmod{q}.$$

Proof. L'égalité implique $(v - v')G = (r' - r)H$. Si $r' = r$, les valeurs seraient congrues ; sinon $r' - r$ est inversible dans le corps d'ordre premier. La formule donne la relation discrète supposée inconnue. □

Proposition 5 (Liaison aux entiers attestés, sans wrap)**.** *Supposons les hypothèses précédentes. Une preuve acceptée dont on extrait (x, d, r_X, r_D) avec $0 \leq x, d < N$ impose $P(a, c)$ pour les valeurs attestées, sauf échec d'extraction ou rupture de liaison.*

Proof. Les points dérivés donnent une ouverture de A à $a' = x + 1$ avec masque r_X, et de C à $c' = x + 1 + d$ avec masque $r_X + r_D$. Si ces valeurs diffèrent modulo q des ouvertures attestées, le lemme précédent donne une rupture de liaison. Sinon $a \equiv a'$ et $c \equiv c' \pmod{q}$. Or $0 \leq a, c \leq U < q$ et $0 \leq a', c' \leq 2^{33} - 1 < q$. Deux représentants dans $[0, q)$ congrus modulo q sont égaux. Donc $a = a'$ et $c = c'$, et l'équivalence entière conclut. □

La borne $U < q$ est essentielle : une autorité attestant un entier sans domaine pourrait signer une valeur $a' + q$ ayant le même scalaire. De même, la signature d'un point fourni aveuglément ne garantit pas que le point représente le montant issu du système métier. Ces risques ne sont pas supprimés par une concordance de tests.

Les signatures d'autorité sont vérifiées hors preuve de plages. Le mode A inclut le challenge complet dans le transcript ; B réutilise une preuve historique et signe une nouvelle présentation sous la clé du détenteur. B ne démontre pas une connaissance fraîche du witness. La composition suppose aussi l'inforgeabilité des signatures, la résistance aux collisions du digest, la séparation des domaines et la comparaison complète du challenge. Aucune probabilité numérique de sécurité n'est déduite des essais.

4 Oracle MATLAB/Octave : exactitude des entiers

L'entrée de `math/circuit/circuit_oracle.m` est un tableau JSON de 1 à 10 000 objets contenant exactement `id`, `category`, `amount` et `cap`. Les valeurs numériques sont des chaînes décimales canoniques, jamais des nombres JSON convertis implicitement en `double`. L'API `jsondecode` documente cette conversion et la normalisation des noms de champs [6]. Des contrôles de forme, de clés, de doublons et d'encodage complètent donc le décodage standard. Un champ `expected` est refusé. L'oracle ne lit aucune réponse attendue dans le corpus.

Lemme 6 (Conversion décimale sans saturation)**.** *Posons* $Q = \lfloor U/10 \rfloor = 1844674407370955161$ *et* $R = 5$*. Pour un préfixe* $v \in [0, U]$ *et un chiffre* $b \in [0, 9]$*, l'opération* $10v + b$ *est représentable si et seulement si* $v < Q$*, ou* $v = Q$ *et* $b \leq R$*.*

Proof. $U = 10Q + R$. Si $v < Q$, $10v + b \leq 10Q - 1 \leq U$. Si $v = Q$, la condition est exactement $b \leq R$. Si $v > Q$, $10v + b \geq 10Q + 10 > U$. L'induction sur les chiffres justifie le contrôle avant multiplication et addition. Q est construit depuis de petits blocs exactement représentables, sans conversion d'un grand littéral flottant. □

Le calcul de x est précédé de $a \geq 1$ et celui de d de $c \geq a$. Cette garde est indispensable : les opérations entières peuvent saturer aux limites des types dans MATLAB et Octave [5, 4]. Une différence négative est rendue par la chaîne vide, pas par zéro ; les autres différences sont rendues exactement, y compris pour un cas refusé.

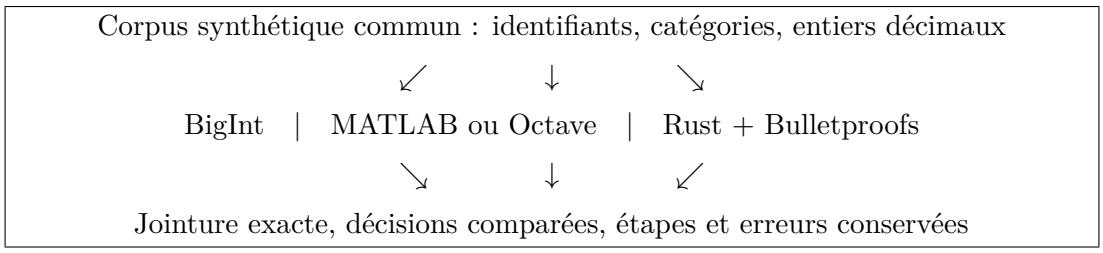
Le formatage décimal évite aussi la division entière au bord du type. Pour une valeur supérieure à $2^{32} - 1$, une division binaire par 10 garde un reste dans $[0, 9]$; avant réduction, ce reste est au plus 19. Le quotient est construit bit par bit. Pour une valeur au plus $2^{32} - 1$, le chemin court utilise `floor(double(v)/10)` : l'entier est représenté exactement et l'erreur d'arrondi du quotient est très inférieure à 0,1, distance minimale d'un quotient non entier au prochain entier. Aucun grand `uint64` n'est envoyé dans ce chemin.

Observation de développement. Sur GNU Octave 7.3.0 dans l'image locale enregistrée, une première version utilisant `idivide` au maximum du type n'a pas satisfait les tests de formatage : $2^{64} - 1$ a été rendu avec une terminaison décimale erronée. Ces essais ont échoué avant toute publication d'un résultat positif. La source livrée n'utilise pas cette division. Ce constat est un résultat de runtime local, pas une conclusion sur toutes les versions MATLAB/Octave.

5 Protocole de corrélation des trois chemins

Pour chaque vecteur i, notons J_i la décision BigInt, M_i celle du runtime numérique réellement exécuté et R_i le résultat Rust approprié. Le corpus et chaque résultat doivent porter des identifiants uniques et être reliés par leurs empreintes. L'égalité des ensembles d'identifiants, des

valeurs et des effectifs est vérifiée *avant* celle des décisions. Une omission, duplication, erreur de processus ou sortie illisible ne vaut jamais un refus métier réussi.



MATLAB et Octave exécutant la même source ne sont pas deux implémentations indépendantes du prédicat. Des chemins concordants peuvent partager une erreur de spécification. La preuve algébrique et les contre-exemples métier restent donc nécessaires.

Classe d'essai	Condition de réussite et sens du résultat
Cas arithmétique valide	$J_i = M_i =$ vrai ; preuves A et B générées, puis acceptées par un processus vérifieur distinct sur les attentes épinglées.
Cas arithmétique invalide	$J_i = M_i =$ faux ; le wrapper Rust peut refuser avant génération. Ce refus n'est pas un test du vérifieur de plages.
Contournement du garde-fou du prover	Construire avec l'API de la bibliothèque une tentative hors plage ; observer séparément erreur de génération ou rejet cryptographique. Ne pas compter une absence de preuve comme une preuve rejetée.
Preuve modifiée	Garder un encodage structurel valide et, lorsque requis, re-signer le bundle de fixture. Le refus doit atteindre l'étape cryptographique revendiquée, au lieu de s'arrêter à une signature extérieure incorrecte.
Contexte ou politique modifiés	Refus d'autorisation distinct d'un échec arithmétique ; compte d'effets inchangé dans les essais applicatifs concernés.

Le vérifieur de campagne ne reçoit ni masques ni clé privée du détenteur. Les montants du corpus sont synthétiques et publics pour comparer les oracles ; cela ne constitue pas un test d'anonymat. Les preuves, métadonnées et engagements restent corrélables. Un processus séparé démontre une frontière d'entrée testée, pas une isolation matérielle ou un audit externe.

6 Couverture, artefacts et interprétation

Les tailles 128, 1 024 et 4 096 sont des paramètres de campagne, pas des résultats présumés. Un rapport doit donner les cas effectivement terminés, leurs catégories, les deux verdicts arithmétiques, le point exact de refus Rust et les preuves réellement vérifiées. Les frontières comprennent 0, 1, $N-1$, N, $N+1$, $2N-1$, $2N$, $2^{53} \pm 1$, U et les relations $c < a$, $c = a$, $c - a = N - 1$, $c - a = N$. Le parsing teste notamment $U + 1$, les zéros initiaux, signes, nombres JSON, champs supplémentaires et identifiants dupliqués.

L'autotest local exécuté sous Octave 7.3.0 le 17 octobre 2026 a vérifié 11 cas de frontière, rejeté 36 entrées invalides et refusé un écrasement de fichier. Il a parcouru les 4 096 paires du carré $[0, 63]^2$, dont 2 016 satisfont P. Cet autotest est numérique ; il n'exécute pas Rust. Le statut d'une campagne corrélée complète est déterminé par ses artefacts, et non par ces nombres ou par la compilation du présent document.

Les fichiers publics prévus comprennent `vectors.json`, `octave.json` ou `matlab.json`, `prover-results.json`, `verifier-results.json`, `correlation.json`, les requêtes publiques de vérification compressées, un rapport HTML et ses figures. Ils sont placés dans un nouveau répertoire d'exécution ; aucune archive antérieure n'est écrasée. Les champs `runtime`, `runtime_version` et `executed_in_matlab` distinguent les moteurs réellement utilisés. Une exécution GitHub Actions ne remplace pas cette attribution ; une action configurée ou un job échoué ne vaut pas une exécution MATLAB réussie.

Les figures doivent dériver de résultats présents et porter effectifs, unités et erreurs. Un panneau sans mesure est marqué « non mesuré ». Les latences locales n'établissent ni SLA ni supériorité à Vega : les relations, contrats d'émission et conditions diffèrent. Aucun résultat GPU n'est déduit de ce test CPU. Des tailles de corpus croissantes ne mesurent pas la probabilité cryptographique de soundness et ne démontrent pas une couverture exhaustive du domaine $\mathcal{D}$.

Les preuves conditionnelles ici sont rédigées et relues, sans assistant de preuve formelle. Les hypothèses de connaissance, de liaison et de signatures restent héritées ; l'autorisation distante exactement une fois, la vérité des sources, les canaux auxiliaires et la qualification bancaire restent hors portée. Un manifeste SHA-256 lie les octets sous une référence de confiance ; sa date locale n'est pas un horodatage tiers et son contenu ne prouve pas la sémantique d'une assertion.

References

[1] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. Cryptology ePrint Archive, Report 2017/1066, 2017. URL https://eprint.iacr.org/2017/1066. Revised 2022-04-14, correcting the Fiat–Shamir section. Consulted 2026-10-17.

[2] Dalek Cryptography. Bulletproofs 5.0.0: Rangeproof api. URL https://docs.rs/bulletproofs/5.0.0/bulletproofs/struct.RangeProof.html. Pinned implementation documentation; consulted 2026-10-17.

[3] Henry de Valence, Jack Grigg, Mike Hamburg, Isis Lovecruft, George Tankersley, and Filippo Valsorda. The ristretto255 and decaf448 groups. RFC 9496, Informational, 2023. URL https://www.rfc-editor.org/rfc/rfc9496.html. Consulted 2026-10-17.

[4] GNU Octave. Gnu octave 7.3.0: Integer arithmetic. URL https://docs.octave.org/v7.3.0/Integer-Arithmetic.html. Version-specific primary manual; consulted 2026-10-17.

[5] The MathWorks, Inc. Integers, . URL https://www.mathworks.com/help/matlab/matlab_prog/integers.html. Integer types, arithmetic and overflow; consulted 2026-10-17.

[6] The MathWorks, Inc. jsondecode: Decode json-formatted text, . URL https://www.mathworks.com/help/matlab/ref/jsondecode.html. Primary API documentation; consulted 2026-10-17. Documentation is not evidence of local MATLAB execution.